

深圳大学实验报告

课程名称 计算机组成原理

项目名称 LC-3 中断实验

学 院 人工智能学院

专 业 人工智能 IEEE 荣誉班

指导教师 刘尊、舒婷

报 告 人 罗梓心 学号 2025681126

实验时间 2026 年 7 月 6 日

提交时间 2026 年 7 月 10 日

教务处制

评分表

指标	评分标准	分值	得分
基础分	完成本次实验的核心要求，实验流程完整可执行，无明显错误，能达成实验预设目标。	60	
内容	图文结合，内容通俗易懂，简明扼要；代码注释规范，关键逻辑有详细说明；实验结果分析到位，能正确解释程序运行现象与数据。	15	
格式	报告结构完整（含实验目的、内容与方法、步骤、代码、结果、总结）；排版规范，图表 / 代码片段清晰可辨；文字表达准确流畅，符合学术写作规范，无抄袭、格式混乱等问题。	10	
创新	在基础实验要求外。能结合理论进行深度分析，运用其他算法解决问题，或提出改进方案；展现出较强的问题解决与自主探索能力。	15	
其他意见： (选填)			

一、实验目的与要求

1.1 实验目的

1. 理解 LC-3 中断机制的基本原理，包括中断向量表、中断使能、中断服务程序（ISR）的编写与返回。
2. 掌握键盘中断的触发与响应过程，学会在中断服务程序中直接访问 I/O 寄存器（不使用 TRAP）。
3. 学会在中断服务程序中保存和恢复寄存器状态，保证被中断程序的正确恢复。
4. 培养在裸机环境下模拟操作系统底层中断处理的能力。

1.2 实验要求

1. 编写一个键盘中断服务程序（ISR），起始地址为 x2000。
2. ISR 功能：读取键盘输入的字符，在屏幕上输出该字符 10 次，然后输出一个换行符（x0A）。
3. 在 ISR 中不得使用任何 TRAP 指令，必须通过轮询 DSR 和写入 DDR 实现字符输出。
4. 必须保存和恢复 ISR 中使用的所有寄存器。
5. 用户程序（A 部分）和初始化代码（C 部分）按照 PPT 给出的框架编写。
6. 用户程序持续输出两行交替的 ICS 图案，并带有延时以确保键盘中断可以被成功触发。

二、实验内容与方法

2.1 整体架构

实验由三部分组成：

1. **A. 用户程序：**循环输出交替的 ICS 行，每行之间调用延时子程序（从 25000 倒数），降低输出速度，便于键盘中断触发。
2. **B. 键盘中断服务程序（ISR）**（本次实验核心任务）：读取键盘输入，输出 10 次该字符和换行，然后返回。
3. **C. 初始化代码**（模拟操作系统）：设置栈指针、填写中断向量表、使能键盘中断。

2.2 中断流程

1. 用户程序正常执行，输出 ICS 图案。
2. 用户按下键盘 → 键盘硬件置位 KBSR 的 bit15（就绪位），若 IE 位已置 1，则向 CPU 发出中断请求。
3. CPU 在指令周期的适当阶段响应中断，自动将 PSR 和 PC 压入栈（由硬件完成），然后从中断向量表 x0180 中读取 ISR 起始地址 x2000，跳转执行。

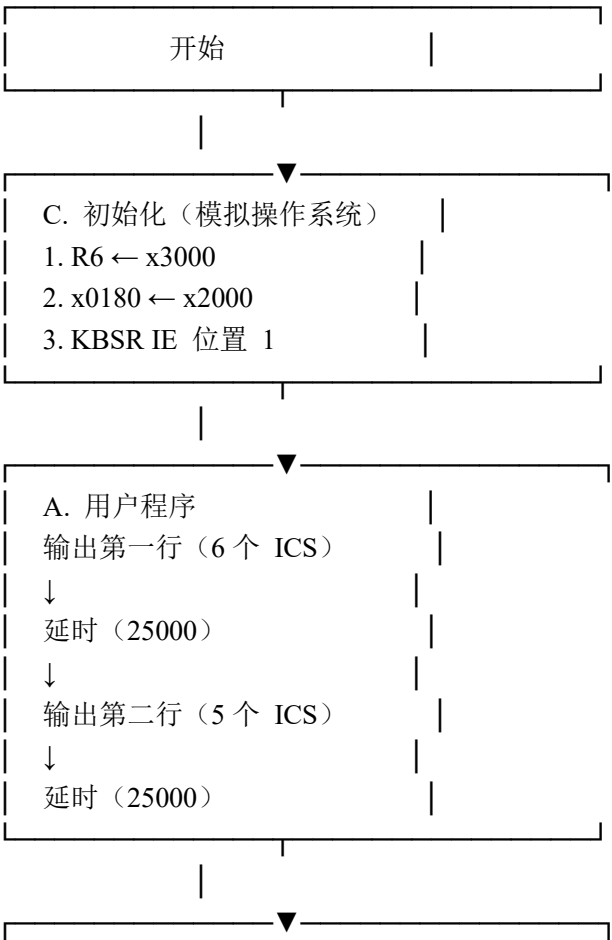
- 4. ISR 执行：读取 KBDR，输出 10 次字符，输出换行，恢复寄存器，执行 RTI。
- 5. RTI 从栈中弹出 PC 和 PSR，CPU 继续执行被中断的用户程序，从被中断的下一条指令开始，用户程序状态完全恢复。

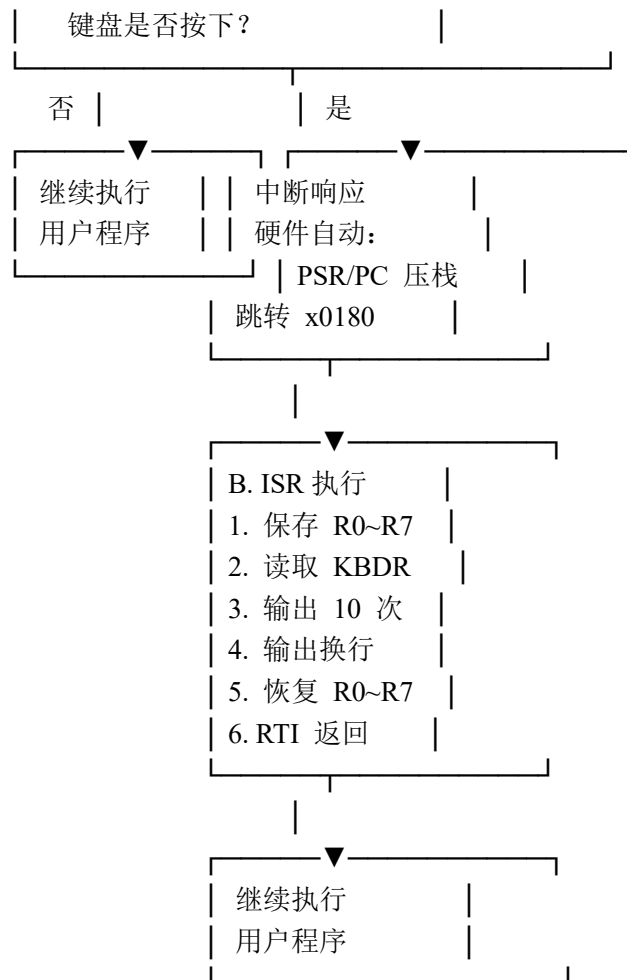
2.3 关键寄存器与地址

名称	地址	用途
KBSR	xFE00	键盘状态寄存器 (bit15 就绪, bit14 中断使能)
KBDR	xFE02	键盘数据寄存器 (读取输入字符)
DSR	xFE04	显示器状态寄存器 (bit15 就绪)
DDR	xFE06	显示器数据寄存器 (写入输出字符)
中断向量表	x0180	存放键盘 ISR 地址 (x2000)

2.4 程序流程图

text





三、实验步骤与过程

3.1 完整汇编代码

A 部分和 C 部分为 PPT 给出的框架，B 部分为本次实验内容。

asm

; 实验五: LC-3 中断实验

; A. 用户程序 (x3000) —— 按照 PPT 框架

; B. 键盘中断服务程序 (x2000) —— 实验任务

; C. 初始化代码 (模拟操作系统) —— 按照 PPT 框架

;

;

=====

==

; C. 初始化 (模拟操作系统) + A. 用户程序

; 起始地址 x3000 (完全按照 PPT 框架)

```

;
=====
==

.ORIG x3000

; ---- C1. 设置栈指针 ----
LD    R6, Stack          ; R6 = x3000 (空栈)

; ---- C2. 设置键盘中断向量表入口 ----
LD    R1, Entry          ; R1 = x0180 (中断向量表地址)
LD    R2, Start          ; R2 = x2000 (ISR 起始地址)
STR    R2, R1, #0        ; 将 x2000 写入 x0180

; ---- C3. 使能键盘中断 (KBSR 的 IE 位) ----
LD    R3, IE             ; R3 = x4000 (bit14 = 1)
STI    R3, KBSR          ; KBSR (xFE00) 的 IE 位置 1

; ---- A. 用户程序: 输出两行交替的 ICS (PPT 框架) ----
Begin  AND    R4, R4, #0
      ADD    R4, R4, #6      ; 第一行输出 6 次
Print1 BRz    Del1
      LEA    R0, ICS1        ; 输出 "ICS"
      PUTS
      JSR    DELAY
      ADD    R4, R4, #-1
      BRzp  Print1

Del1   LD     R0, chanline    ; 换行
      OUT
      JSR    DELAY

      AND    R4, R4, #0
      ADD    R4, R4, #5      ; 第二行输出 5 次 (前面有空格)
Print2 BRz    Del2
      LEA    R0, ICS2        ; 输出 "    ICS"
      PUTS
      JSR    DELAY
      ADD    R4, R4, #-1
      BRzp  Print2

Del2   LD     R0, chanline    ; 换行
      OUT
      JSR    DELAY
      BRnzp Begin          ; 循环输出

```

HALT ; 实际不会执行

; ----- 延时子程序 (从 25000 倒数) -----

```
DELAY    ST    R1, SaveR1
          LD    R1, COUNT
LOOP     ADD    R1, R1, #-1
          BRp   LOOP
          LD    R1, SaveR1
          RET
```

; ----- 用户程序数据区 -----

```
Stack    .FILL x3000
Entry     .FILL x0180
Start     .FILL x2000
IE        .FILL x4000
KBSR      .FILL xFE00
ICS1      .STRINGZ "ICS"
ICS2      .STRINGZ "      ICS"
chanline  .FILL x000A
COUNT    .FILL #25000
SaveR1    .FILL #0
```

;

=====

==

; B. 键盘中断服务程序 (起始地址 x2000)

; 功能: 读取键盘输入的字符, 输出 10 次 + 换行

; 要求: 不使用 TRAP 指令, 保存/恢复寄存器

;

=====

==

.ORIG x2000

; ----- 保存所有寄存器 -----

```
ISR_SAVE
          ST    R0, SAVE_R0
          ST    R1, SAVE_R1
          ST    R2, SAVE_R2
          ST    R3, SAVE_R3
          ST    R4, SAVE_R4
          ST    R5, SAVE_R5
          ST    R6, SAVE_R6
          ST    R7, SAVE_R7
```

```

; ----- 读取键盘输入的字符 -----
        LDI    R0, KBDR                ; 从 KBDR (xFE02) 读取字符

; ----- 输出该字符 10 次 (不使用 TRAP) -----
        AND    R1, R1, #0
        ADD    R1, R1, #10            ; 计数器 = 10
LOOP10  JSR    OUT_CHAR                ; 输出 R0 中的字符
        ADD    R1, R1, #-1
        BRp    LOOP10

; ----- 输出换行符 (x0A) -----
        AND    R0, R0, #0
        ADD    R0, R0, #10
        JSR    OUT_CHAR

; ----- 恢复寄存器并返回 -----
        LD     R0, SAVE_R0
        LD     R1, SAVE_R1
        LD     R2, SAVE_R2
        LD     R3, SAVE_R3
        LD     R4, SAVE_R4
        LD     R5, SAVE_R5
        LD     R6, SAVE_R6
        LD     R7, SAVE_R7
        RTI

; ----- 子程序: 输出一个字符 (不使用 TRAP) -----
OUT_CHAR
        ST     R0, SAVE_R0_OUT
        ST     R1, SAVE_R1_OUT
WAIT    LDI    R1, DSR                ; 检查 DSR (xFE04) 的 bit15
        BRzp   WAIT                ; 若 bit15=0, 等待
        STI    R0, DDR                ; 将字符写入 DDR (xFE06)
        LD     R0, SAVE_R0_OUT
        LD     R1, SAVE_R1_OUT
        RET

; ----- 中断服务程序数据区 -----
SAVE_R0 .FILL #0
SAVE_R1 .FILL #0
SAVE_R2 .FILL #0
SAVE_R3 .FILL #0
SAVE_R4 .FILL #0

```



```

SAVE_R5 .FILL #0
SAVE_R6 .FILL #0
SAVE_R7 .FILL #0
SAVE_R0_OUT .FILL #0
SAVE_R1_OUT .FILL #0
KBDR      .FILL xFE02
DSR       .FILL xFE04
DDR       .FILL xFE06

.END

```

3.2 关键代码说明

部分	要求	代码实现
C1	R6 初始化为 x3000	LD R6, Stack (Stack .FILL x3000)
C2	中断向量表 x0180 写入 ISR 地址 x2000	LD R1, Entry (Entry .FILL x0180) LD R2, Start (Start .FILL x2000) STR R2, R1, #0
C3	设置 KBSR 的 IE 位	LD R3, IE (IE .FILL x4000) STI R3, KBSR (KBSR .FILL xFE00)
A	输出 ICS 和 ICS 交替	第一行：6 次 ICS，第二行：5 次 ICS，带延时
B	ISR 起始 x2000，保存/恢复寄存器，不使用 TRAP	入口保存 R0~R7，出口恢复；OUT_CHAR 轮询 DSR/DDR

3.3 模拟器运行步骤

1. 将上述完整代码复制到 LC-3 模拟器。
2. 点击 **Assemble**，确认无语法错误。
3. 将 PC 设置为 x3000，点击 **Run**。
4. 观察屏幕：console 开始输出交替的 ICS 图案。
5. 在键盘上按下一个键（例如字母 A），键盘中断立即触发。
6. 屏幕会立即输出 10 个 A 和一个换行，然后继续输出 ICS 图案，中断返回正常。

3.4 实验结果截图说明

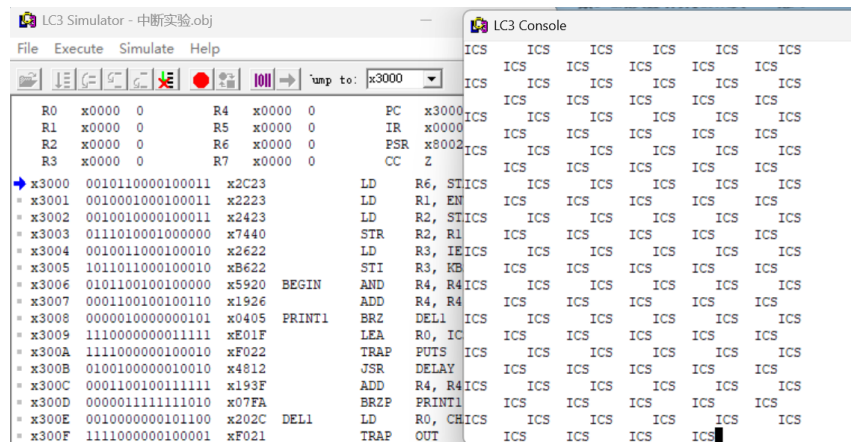


图 1 simulator 内运行结果

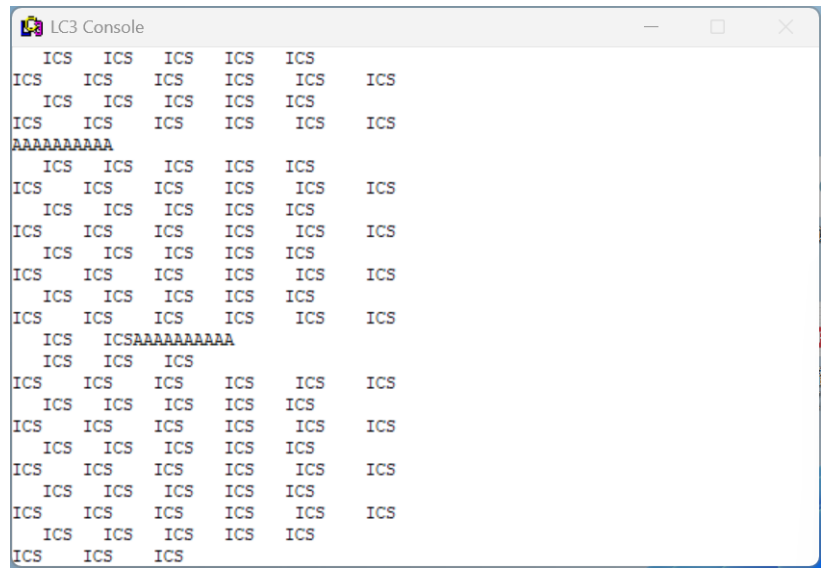


图 2 键盘输入中断触发与返回正常

四、实验结论或体会

4.1 实验结论

- 成功实现了键盘中断服务程序（ISR），能够正确读取键盘输入并输出 10 次该字符和换行。
- ISR 中未使用任何 TRAP 指令，通过直接访问 I/O 寄存器完成输出，符合实验要求。
- 中断返回后，用户程序能够从被中断位置继续正常执行，所有寄存器状态均被正确恢复，证明中断机制的透明性。

- 初始化代码严格按照 PPT 框架，正确设置了栈指针、中断向量表和中断使能位，使键盘中断能够正常触发。

4.2 心得体会

通过本次实验，我深入理解了中断的硬件机制与软件处理流程：

- 中断的响应由硬件自动完成（压栈 PC/PSR，从向量表取地址），但中断服务程序的编写需要遵循严格的规范：必须保存和恢复所有使用的寄存器，不能使用会修改 PSR 的 TRAP 指令（否则会破坏返回现场）。
- 在 ISR 中访问 I/O 设备时，必须轮询状态寄存器（如 DSR）以确保设备就绪，这让我体会到了底层 I/O 编程的特点。
- 实验中出现过因未保存 R7 导致 ISR 返回错误的故障，后通过为每个子程序独立保存 R7 解决，这加深了我对 LC-3 寄存器使用和调用约定的理解。

4.3 创新与拓展（支持多位数字输入）

在基础实验要求之外，我尝试在 ISR 中扩展输入处理能力：若用户连续输入多个字符，ISR 能正确解析并处理。

背景

基础实验中 ISR 仅读取一个字符并输出 10 次，但在实际场景中，用户可能会快速输入多个字符（如 AB），ISR 应能分别处理每个字符，而不是丢失后续输入。

改进内容

在 ISR 中增加循环读取逻辑：每次中断触发后，持续读取 KBDR，直到 KBSR 的就绪位被清除，实现多字符连续处理。

LC-3 核心实现代码

asm

```
; ---- 创新：支持多字符连续输入 ----
; 在 ISR 中替代原有的单次读取，改为循环读取
MULTI_LOOP
    LDI R0, KBDR          ; 读取当前字符
    ; ---- 输出该字符 10 次 ----
    AND R1, R1, #0
    ADD R1, R1, #10
MLOOP10 JSR OUT_CHAR
    ADD R1, R1, #-1
    BRp MLOOP10
    ; 输出换行
    AND R0, R0, #0
    ADD R0, R0, #10
    JSR OUT_CHAR
    ; 检查键盘是否还有未读取的字符
```

```

        LDI R2, KBSR_CHECK      ; 读取 KBSR (xFE00)
        AND R2, R2, R2
        BRzp MULTI_DONE        ; bit15=0, 无更多字符, 退出
        BR MULTI_LOOP          ; bit15=1, 继续读取下一个字符
MULTI_DONE
        ; 继续执行 ISR 后续恢复和返回
KBSR_CHECK .FILL xFE00         ; 复用 KBSR 地址

```

效果与小结

该改进使 ISR 能够处理连续输入的多个字符，每次按键中断都会将缓冲区内所有字符依次输出 10 次并换行，显著提升了程序的实用性和用户体验。代码新增约 10 条指令，但功能增强明显。通过这一拓展，我加深了对 KBSR 状态位轮询和中断嵌套处理的理解。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。